

- [Manual Práctico de PlantUML](#)
 - [1. INTRODUCCIÓN](#)
 - [1.1 Propósito del manual](#)
 - [Instalación y visualización](#)
 - [Diagrama de casos de uso](#)
 - [Casos de uso](#)
 - [Actores](#)
 - [Cambiar el estilo del actor](#)
 - [Descripción de Casos de uso](#)
 - [Utilice el paquete](#)
 - [Ejemplo básico](#)
 - [Extensión](#)
 - [Usando notas](#)
 - [Estereotipos](#)
 - [Cambio de dirección de las flechas](#)
 - [Dividiendo los diagramas](#)
 - [Dirección: de izquierda a derecha](#)

Manual Práctico de PlantUML

1. INTRODUCCIÓN

¿Qué es PlantUML? [PlantUML](#) es una herramienta muy versátil que facilita la creación rápida y directa de una amplia gama de diagramas.

Utilizando un lenguaje sencillo e intuitivo, los usuarios pueden redactar sin esfuerzo diversos tipos de diagramas. Para una exploración detallada de las capacidades del lenguaje y la sintaxis, [consulta la Guía de Referencia del Lenguaje PlantUML](#).

Al ser nuevos usando PlantUML, lo más recomendable es que comiences con su [página de inicio](#) para ponerte en marcha rápidamente. Si tienes alguna pregunta, su [página F.A.Q.](#) es un recurso valioso. Además, PlantUML se puede integrar perfectamente con una variedad de otras herramientas para mejorar su flujo de trabajo.

¿Por qué este manual?

Característica	Manual Tradicional	Este Manual
Enfoque	Teórico	100% Práctico
Requisitos	Conocimiento UML	Cero experiencia
Resultados	En capítulos finales	Desde el minuto 1
Personalización	Limitada	Skins y templates

1.1 Propósito del manual

Objetivo principal: Convertirte en productor de diagramas técnicos en 4 horas, dominando:

@startuml Objetivos

@enduml

Instalación y visualización

1. **Online** La forma más sencilla de visualizar un diagrama en PlantUML es ejecutarlo desde el [servidor web oficial](#).
2. **VS Code** Para instalar PlantUML y usarlo en Visual Studio Code (VSCode), sigue estos pasos: **Requisitos previos** Java Runtime Environment (JRE):
 - PlantUML requiere Java para funcionar.
 - Descárgalo de Oracle JDK o OpenJDK.

Verifica que esté instalado ejecutando en la terminal:

```
java -version
```

Instalación en VSCode:

- Abre VSCode y ve a Extensiones (Ctrl+Shift+X).
- Busca e instala la extensión: "PlantUML" (por jebbs.plantuml).

Configuración básica

Habilitar la vista previa:

- Abre un archivo .puml o .plantuml.

- Presiona Alt + D (Windows/Linux) o Option + D (Mac) para abrir la vista previa.
- (Opcional) Configurar Graphviz (para diagramas complejos):
- [Descarga Graphviz](#). desde graphviz.org.
- Añade la ruta de dot (Graphviz) en la configuración de VSCode, igualmente en la instalación puedes marcar la opción de añadirlo al path / variables de entorno.

Ejemplo de uso:

- Crea un archivo diagrama.puml.
- Escribe un diagrama sencillo:

```
@startuml
Alice -> Bob: Hola
Bob --> Alice: ¿Cómo estás?
@enduml
```

```
@startuml
Alice -> Bob: Hola
Bob --> Alice: ¿Cómo estás?
@enduml
```

Visualización:

Para visualizar online se debe presionar el botón **submit**, en cambio para visualizar desde vscode se debe presionar el shortcut **Alt+D**

Diagrama de casos de uso

Un **diagrama de casos de uso**. es una representación visual utilizada en ingeniería de software para representar las interacciones entre **los actores** del sistema y el propio sistema. Captura el comportamiento dinámico de un sistema ilustrando sus casos de uso y los roles que interactúan con ellos. Estos diagramas son esenciales para especificar los **requisitos funcionales** del sistema y comprender cómo interactuarán los usuarios con él. Al proporcionar una visión de alto nivel, los diagramas de casos de uso ayudan a las partes interesadas a comprender la funcionalidad del sistema y su valor potencial.

PlantUML ofrece un enfoque único para crear diagramas de casos de uso a través de su lenguaje basado en texto. Una de las principales ventajas de utilizar PlantUML es su **sencillez y eficacia**. En lugar de dibujar manualmente formas y conexiones, los usuarios pueden definir sus diagramas utilizando descripciones textuales intuitivas y concisas. Esto no sólo acelera el proceso de creación de diagramas, sino que también garantiza su **coherencia y precisión**. La capacidad de integrarse con varias plataformas de documentación y su amplia gama de formatos de salida compatibles hacen de PlantUML una herramienta versátil tanto para desarrolladores como para no desarrolladores. Por último, al ser de **código abierto**, PlantUML cuenta con una [sólida comunidad](#) que contribuye continuamente a su mejora y ofrece una gran cantidad de recursos para usuarios de todos los niveles.

Casos de uso

Los casos de uso se encierran entre paréntesis (porque dos paréntesis parecen un óvalo).

También puede utilizar la palabra clave **usecase** para definir un caso de uso . Y puede definir un alias, utilizando la palabra clave **as**. Este alias se utilizará más adelante, cuando se definan las relaciones.

```
@startuml

(primer usecase)
(otro usecase) as (UC2)
usecase UC3
usecase (último\nusecase) as UC4

@enduml
```

```
@startuml

(primer usecase)
(otro usecase) as (UC2)
usecase UC3
usecase (último\nusecase) as UC4

@enduml
```

[Consultar visualización online](#)

Actores

Los actores se encierran entre dos puntos. También puedes usar la palabra reservada **actor** para definir un actor. Además puedes definir un alias, usando la palabra reservada **as**. Este alias será usado más adelante, cuando definamos relaciones. Veremos más adelante que las declaraciones de los actores son opcionales.

Ejemplo en PlantUML:

```
@startuml

:Primer Actor:
:otro\nactor: as Men2
actor tipo3
actor :ultimo actor: as tipo4

@enduml
```

```
@startuml

:Primer Actor:
:otro\nactor: as Men2
actor tipo3
actor :ultimo actor: as tipo4

@enduml
```

[Consultar visualización online](#)

Cambiar el estilo del actor

Puedes cambiar el estilo del actor de hombre==== palo ==== (==== por defecto ====) a: un hombre impresionante con el comando **skinparam actorStyle awesome**; un hombre hueco con el comando **skinparam actorStyle hollow**.

Hombre palo (por defecto):

```
@startuml

:usuario: --> (Use)
"administrador principal" as Admin
"Usa la aplicación" as (Use)
```

```
Admin --> (administra la aplicación)
@enduml
```

```
@startuml
:usuario: --> (Use)
"administrador principal" as Admin
"Usa la aplicación" as (Use)
Admin --> (administra la aplicación)
@enduml
```

[Consultar visualización online](#)

Hombre impresionante:

```
@startuml
skinparam actorStyle awesome
:usuario: --> (Use)
"administrador principal" as Admin
"Usa la aplicación" as (Use)
Admin --> (administra la aplicación)
@enduml
```

```
@startuml
skinparam actorStyle awesome
:usuario: --> (Use)
"administrador principal" as Admin
"Usa la aplicación" as (Use)
Admin --> (administra la aplicación)
@enduml
```

[Consultar visualización online](#)

Hombre hueco:

```
@startuml
skinparam actorStyle Hollow
:usuario: --> (Use)
"administrador principal" as Admin
"Usa la aplicación" as (Use)
Admin --> (administra la aplicación)
@enduml
```

```
@startuml
skinparam actorStyle Hollow
:usuario: --> (Use)
"administrador principal" as Admin
"Usa la aplicación" as (Use)
Admin --> (administra la aplicación)
@enduml
```

[Consultar visualización online](#)

Descripción de Casos de uso

Si quiere realizar una descripción en varias líneas, puede usar citas (" ").

También puede usar los siguientes separadores: -- .. == ___. Y puede introducir títulos dentro de los separadores.

```
@staruml

usecase UC1 as "Puedes usar
varias líneas para definir tu caso de uso.
También puedes usar separadores.
--
Se permiten varios tipos de separadores.
==
Y puedes agregar títulos:
..En Conclusion..
Esto permite descripciones largas.."

@enduml
```

```
@startuml

usecase UC1 as "Puedes usar
varias líneas para definir tu caso de uso.
También puedes usar separadores.
--
Se permiten varios tipos de separadores.
==
Y puedes agregar títulos:
..En Conclusion..
Esto permite descripciones largas.."

@enduml
```

Utilice el paquete

Puede utilizar paquetes para agrupar actores o casos de uso

```
@startuml
left to right direction
actor Invitado as i
package Profesional {
    actor Chef as c
    actor "Crítico de Comida" as cc
}
package Restaurante {
    usecase "Come" as UC1
    usecase "Paga" as UC2
    usecase "Bebe" as UC3
    usecase "Evalua" as UC4
}
cc --> UC4
i --> UC1
g --> UC2
g --> UC3
@enduml
```

```
@startuml
left to right direction
actor Invitado as i
package Profesional {
    actor Chef as c
    actor "Crítico de Comida" as cc
}
package Restaurante {
    usecase "Come" as UC1
    usecase "Paga" as UC2
    usecase "Bebe" as UC3
    usecase "Evalua" as UC4
}
cc --> UC4
i --> UC1
g --> UC2
g --> UC3
@enduml
```

```
@startuml
left to right direction
actor "Critico de comida" as cc
rectangle Restaurante {
    usecase "Come" as UC1
    usecase "Paga" as UC2
    usecase "Bebe" as UC3
}
cc --> UC1
cc --> UC2
cc --> UC3
@enduml
```

```
@startuml
left to right direction
actor "Critico de comida" as cc
rectangle Restaurante {
    usecase "Come" as UC1
    usecase "Paga" as UC2
    usecase "Bebe" as UC3
}
cc --> UC1
cc --> UC2
cc --> UC3
@enduml
```

[Consultar visualización online](#)

Ejemplo básico

Para relacionar actores y casos de uso, la flecha `-->` es usada. Cuanto más guiones `-` en la flecha, más larga será la misma. Puedes añadir una etiqueta en la flecha, añadiendo el carácter `:` en la definición de la flecha. En este ejemplo, puedes ver que User no ha sido definido, y es usado como un actor.

```
@startuml

User -> (Inicia)
User --> (Usa la aplicación) : Una pequeña descripción

:Main Admin: ---> (Usa la aplicación) : Esto es\nOtra\nDescripción

@enduml
```

```
@startuml

User -> (Inicia)
User --> (Usa la aplicación) : Una pequeña descripción

:Main Admin: ---> (Usa la aplicación) : Esto es\nOtra\nDescripción

@enduml
```

[Consultar visualización online](#)

Extensión

Si un actor/caso de uso extiende a otro, puede utilizar el símbolo **<|--**.

```
@startuml
:Main Admin: as Administrador
(Usa la aplicación) as (Use)

Usuario <|-- Administrador
(Inicia) <|-- (Use)

@enduml
```

```
@startuml
:Administrador Principal: as Administrador
(Usa la aplicación) as (Usa)

Usuario <|-- Administrador
(Inicia) <|-- (Usa)

@enduml
```

[Consultar visualización online](#)

Usando notas

Puedes usar las palabras claves: **note left of** , **note right of** , **note top of** , **note bottom of**, para añadir notas relacionadas a un objeto en particular. También se puede añadir una nota solitaria con la palabra clave **note**, y después relacionarla con otro objeto usando el símbolo **...**

```
@startuml
:Administrador Principal: as Administrador
(Usa la aplicación) as (Usa)

Usuario -> (Inicia)
Usuario --> (Usa)

Administrador ---> (Usa)

note right of Admin : Esto es un ejemplo.

note right of (Usa)
  Una nota nota también puede
  tener varias líneas
end note

note "Esta nota está conectando\nvarios objetos." as N2
(Inicia) .. N2
N2 .. (Usa)
@enduml
```

```
@startuml
:Administrador Principal: as Administrador
(Usa la aplicación) as (Usa)

Usuario -> (Inicia)
Usuario --> (Usa)

Administrador ---> (Usa)

note right of Administrador : Esto es un ejemplo.

note right of (Usa)
  Una nota nota también puede
  tener varias líneas
end note

note "Esta nota está conectando\nvarios objetos." as N2
(Inicia) .. N2
N2 .. (Usa)
@enduml
```

[Consultar visualización online](#)

Estereotipos

Puedes añadir estereotipos mientras defines actores y casos de uso, usando << y >>.

```
@startuml
Usuario << Humano >>
:base de datos Principal: as MySql << Aplicación >>
(Inicia) << Un intento >>
(Usa la aplicación) as (Usa) << Principal >>

Usuario -> (Inicia)
Usuario --> (Usa)

MySql --> (Usa)

@enduml
```

```
@startuml
Usuario << Humano >>
:base de datos Principal: as MySql << Aplicación >>
(Inicia) << Un intento >>
(Usa la aplicación) as (Usa) << Principal >>

Usuario -> (Inicia)
Usuario --> (Usa)

MySql --> (Usa)

@enduml
```

[Consultar visualización online](#)

Cambio de dirección de las flechas

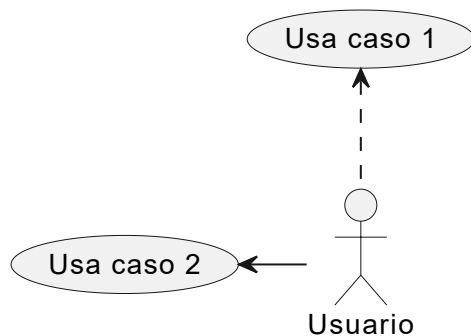
Por defecto, los enlaces entre clases tienen dos guiones `--` y están orientados verticalmente. Es posible utilizar un enlace horizontal poniendo un solo guión (o punto) como este

```
@startuml
:Usuario: --> (Usa caso 1)
:Usuario: -> (Usa caso 2)
@enduml
```

```
@startuml
:Usuario: --> (Usa caso 1)
:Usuario: -> (Usa caso 2)
@enduml
```

[Consultar visualización online](#)

También se puede cambiar la dirección invirtiendo el enlace:



```
@startuml
@startuml
(Usa caso 1) <.. :Usuario:
(Usa caso 2) <- :Usuario:
@enduml
```

[Consultar visualización online](#)

También es posible cambiar la dirección de la flecha añadiendo las palabras clave **left**, **right**, **up** o **down** dentro de la flecha

```
@startuml
:Usuario: -left-> (Izquierda)
:Usuario: -right-> (Derecha)
:Usuario: -up-> (Arriba)
:Usuario: -down-> (Abajo)
@enduml
```

```
@startuml
:Usuario: -left-> (Izquierda)
:Usuario: -right-> (Derecha)
:Usuario: -up-> (Arriba)
:Usuario: -down-> (Abajo)
@enduml
```

[Consultar visualización online](#)

Puede acortar la flecha utilizando sólo el primer carácter de la dirección (por ejemplo, -d- en lugar de -down-) o los dos primeros caracteres (-do-). Tenga en cuenta que no

debe abusar de esta funcionalidad : Graphviz suele dar buenos resultados sin retoques. Y con el **eft to right direction** parámetro

```
@startuml
left to right direction
:user: -left-> (dummyLeft)
:user: -right-> (dummyRight)
:user: -up-> (dummyUp)
:user: -down-> (dummyDown)
@enduml
```

Dividiendo los diagramas

La palabra clave **newpage** divide su diagrama en varias páginas o imágenes. **@startuml**

```
:actor1: --> (Usecase1)
newpage
:actor2: --> (Usecase2)
@enduml
```

```
@startuml
:actor1: --> (Usecase1)
newpage
:actor2: --> (Usecase2)
@enduml
```

Dirección: de izquierda a derecha

El comportamiento general cuando se construye un diagrama, es **top to bottom**.

```
@startuml
'default
top to bottom direction
user1 --> (Usecase 1)
user2 --> (Usecase 2)

@enduml
```

```
@startuml
'default
top to bottom direction
user1 --> (Usecase 1)
user2 --> (Usecase 2)

@enduml
```

Puede cambiar a **left to right** usando el comando **left to right direction**. En ocasiones, el resultado es mejor con esta dirección.

```
@startuml

left to right direction
user1 --> (Usecase 1)
user2 --> (Usecase 2)

@enduml
```

```
@startuml

left to right direction
user1 --> (Usecase 1)
user2 --> (Usecase 2)

@enduml
```

Hay más objetos que se pueden modificar dentro de Plant UML, sin embargo esto es lo que esencialmente necesitamos para la clase. Es usted libre de explorar su documentación y experimentar.